

PCAP Traffic Analysis Report

Investigation of Suspicious Network Activity in maccdc2011 PCAP

May 18, 2026

1 Introduction

This paper presents a technical analysis of the unusual traffic detected in the file `maccdc2011_00010_20110312194033.pcap`. The analysis was performed from the perspective of an analyst in a Security Operations Centre. The objective of the study was to determine whether there was any potential security threat based on the information from the network captures.

Since the initial PCAP file was relatively large, it was broken down into smaller files depending on the type of protocols such as DNS, FTP, HTTP, malformed packets, SMB, and SYN scanning. This was useful for the project, in addition to providing a means of verifying any findings through the original packet capture file. TCP, HTTP, and FTP are the three protocols that played an important role in this project as they clearly indicated probing activities, web application scanning, and abuse of services.

Some of the most interesting results from this exercise include TCP SYN scanning from `192.168.201.74`, automated web application scanning from `192.168.201.72` to `192.168.22.138`, probing of login and transactions endpoints for the MyBanco web application, and anonymous FTP activity between `192.168.203.248` and `192.168.23.252`. From the findings presented above, it is evident that the approach used in the attacks was multi-faceted and included steps such as reconnaissance and enumeration of vulnerabilities.

2 Tools and Methodology

Wireshark was chosen to be the primary analyzer tool due to its capability to perform thorough packet analysis, protocol dissection, display filtering, TCP stream reassembly, and ability to examine application layer information (Wireshark Foundation 2025). This

was crucial because the task at hand required not only recognition of protocols but also extraction of some useful information like HTTP request path, form submission data, response code, User-Agent string, FTP command list, and even clear-text password.

Wireshark proved itself to be very helpful during our investigation by allowing certain suspicious activity to be distinguished through use of appropriate display filters. For instance, a filter for all SYN packets that did not have an ACK flag enabled helped identify SYN scanning activity. In order to isolate HTTP traffic between the web attacker in question and the web server, IP address based filters were utilized, while certain web attack attempts were identified using HTTP method and payload based filters. The "Follow TCP Stream" feature was used to combine all request and response messages thus making sure the packet represented full communication session.

NetworkMiner was also considered as a supporting tool for forensics. NetworkMiner is helpful since it allows you to have a host-oriented analysis of your PCAP files and helps identify hosts, sessions, passwords, and extractions (Netresec 2025). Wireshark was the tool for submitting evidence because it provided a clear understanding of packet fields, filters, and streams needed for replication purposes.

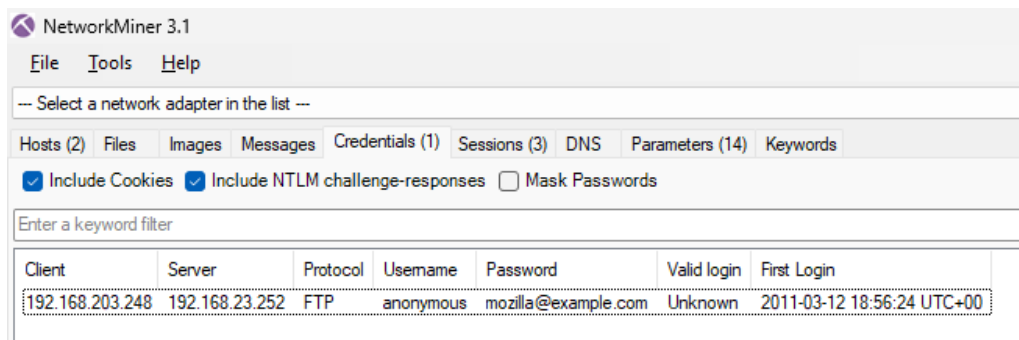


Figure 1: NetworkMiner Credentials view showing FTP authentication artifacts extracted from the exported FTP traffic between 192.168.203.248 and 192.168.23.252.

There was a particular methodology that was followed in order to achieve results. First, the idea was to classify packets based on protocols. Then, the evaluation of host pairs and protocols were undertaken to see if there was an anomaly in their communication. This process involved applying specific filters in Wireshark such as SYN scan, HTTP enumeration, HTTP post, put, transaction, and FTP authentication.

Key filters used during the investigation included:

```
ip.src == 192.168.201.74 && tcp.flags.syn == 1 && tcp.flags.ack == 0
```

```
ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 && http.request
```

```
ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 &&  
http.request.method == "PUT"
```

```
ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 &&  
http contains "cmd.exe"
```

```
ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 &&  
http.request.method == "POST"
```

```
ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 &&  
http contains "doTransaction"
```

```
ftp.request.command == "USER" || ftp.request.command == "PASS"
```

```
ip.addr == 192.168.203.248 && ip.addr == 192.168.23.252 && ftp
```

These filters support reproducibility because another analyst can apply the same filters to the same PCAP and locate the same evidence.

3 Network Traffic Identification

The most relevant hosts obtained from the analysis are presented in Table 1. The types of network traffic observed include scanning, HTTP web application, and FTP authentication traffic. The most suspicious host was 192.168.201.72, which regularly made connections with the web server at 192.168.22.138. Another suspicious host was 192.168.201.74,

Table 1: Key hosts identified during PCAP analysis

Host	Role	Protocol	Observed behaviour
192.168.201.74	Suspected scanner	TCP	SYN scanning against internal hosts and ports
192.168.201.72	Suspected web attacker/scanner	HTTP	Web enumeration, login attempts, payload testing, transaction endpoint testing
192.168.22.138	Web server target	HTTP	Apache/Ubuntu server running MyBanco Customer Frontend
192.168.203.248	FTP client	FTP	Anonymous FTP login attempts
192.168.23.252	FTP server	FTP	Accepted anonymous FTP authentication

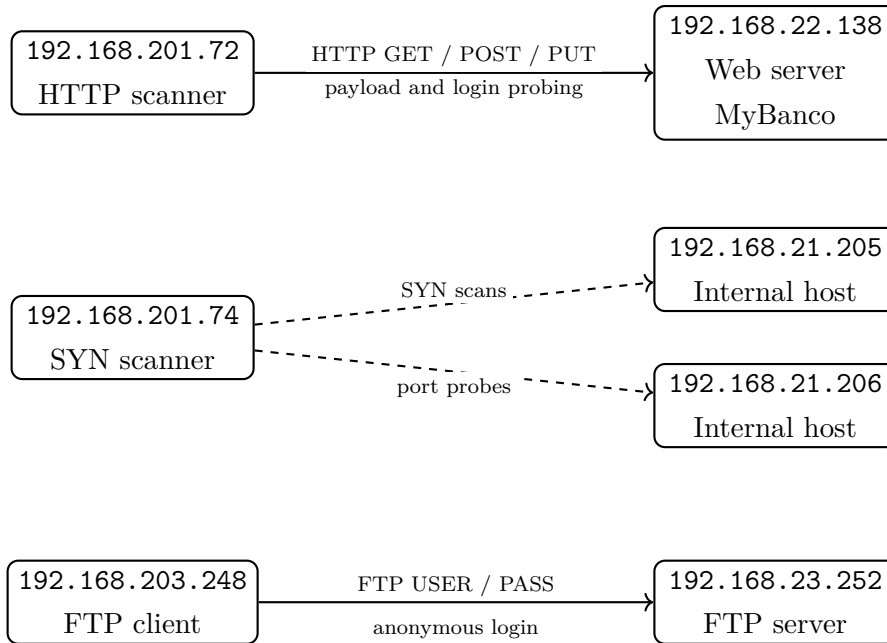


Figure 2: Simplified host communication map showing the main suspicious relationships identified in the PCAP.

The web server under attack declared itself to be **Apache/2.2.14 (Ubuntu)**. In some cases, the HTTP response headers included an extra **X-Powered-By** parameter of **MyBanco Customer Frontend v0.09**. The fact that the attacker interacted with a web application instead of a regular web server is revealed by the above facts. The seriousness of the attack

is accentuated by the fact that login and transfer operations were among the endpoints being tested.

4 Findings

4.1 Finding 1: TCP SYN Scanning

The first one noticed was the SYN scan performed by TCP from the IP address 192.168.201.74. The filter used for the detection was `ip.src == 192.168.201.74 && tcp.flags.syn == 1 && tcp.flags.ack == 0`, which identified multiple SYN packets sent to various ports on internal hosts. The strength of this type of scanning lies in the ability to detect live hosts and open ports without establishing full connection.

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	Flows
192.168.201.74	33741	192.168.21.205	384	1	64 bytes	492858	1	100.00%	1	64 bytes	0	0 bytes	739.955253	0.0000			0
192.168.201.74	33741	192.168.21.205	389	1	64 bytes	498077	1	100.00%	1	64 bytes	0	0 bytes	751.632033	0.0000			0
192.168.201.74	33741	192.168.21.205	407	1	64 bytes	511910	1	100.00%	1	64 bytes	0	0 bytes	784.234872	0.0000			0
192.168.201.74	33741	192.168.21.205	443	1	64 bytes	513296	1	100.00%	1	64 bytes	0	0 bytes	787.612543	0.0000			0
192.168.201.74	33741	192.168.21.205	445	1	64 bytes	527518	1	100.00%	1	64 bytes	0	0 bytes	817.940790	0.0000			0
192.168.201.74	33740	192.168.21.206	1	1	64 bytes	255933	1	100.00%	1	64 bytes	0	0 bytes	280.677739	0.0000			0
192.168.201.74	33740	192.168.21.206	7	1	64 bytes	262569	1	100.00%	1	64 bytes	0	0 bytes	283.867691	0.0000			0
192.168.201.74	33740	192.168.21.206	9	1	64 bytes	286078	1	100.00%	1	64 bytes	0	0 bytes	314.994557	0.0000			0
192.168.201.74	33740	192.168.21.206	13	1	64 bytes	289328	1	100.00%	1	64 bytes	0	0 bytes	318.082353	0.0000			0
192.168.201.74	33740	192.168.21.206	21	1	64 bytes	300355	1	100.00%	1	64 bytes	0	0 bytes	327.514147	0.0000			0

Figure 3: TCP SYN scan traffic from 192.168.201.74 to multiple internal hosts and ports.

Figure 3 presents 192.168.201.74 communicating with internal hosts like 192.168.21.205 and 192.168.21.206 on multiple ports. The packets were quite small, single-packet connections with the SYN flag on and ACK off. It was clearly visible that this was a scan for reconnaissance purposes rather than regular client-server.

4.2 Finding 2: HTTP GET Enumeration

There was considerable amount of HTTP traffic seen between 192.168.201.72 and 192.168.22.138. In addition, there were numerous attempts made to get certain files or extensions. Some examples of file types that may be used with the name `sfi9876` are `.manifest`, `.csproj`, `.pht`, `.ora`, `.css`, `.log`, and `.dll`.

ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 && http.request						
No.	Time	Source	Destination	Protocol	Length	Info
588677	193.454230	192.168.201.72	192.168.22.138	HTTP	247	GET / HTTP/1.1
588684	193.461270	192.168.201.72	192.168.22.138	HTTP	254	GET /sfi9876 HTTP/1.1
588696	193.465015	192.168.201.72	192.168.22.138	HTTP	255	GET /sfi9876/ HTTP/1.1
588699	193.466313	192.168.201.72	192.168.22.138	HTTP	255	GET /~sfi9876 HTTP/1.1
588717	193.468510	192.168.201.72	192.168.22.138	HTTP	257	GET /sfi9876.7z HTTP/1.1
588718	193.468560	192.168.201.72	192.168.22.138	HTTP	263	GET /sfi9876.manifest HTTP/1.1
588719	193.468609	192.168.201.72	192.168.22.138	HTTP	261	GET /sfi9876.csproj HTTP/1.1
588720	193.468659	192.168.201.72	192.168.22.138	HTTP	259	GET /sfi9876.part HTTP/1.1
588721	193.468710	192.168.201.72	192.168.22.138	HTTP	258	GET /sfi9876.ppt HTTP/1.1
588722	193.468759	192.168.201.72	192.168.22.138	HTTP	258	GET /sfi9876.ora HTTP/1.1
588723	193.468762	192.168.201.72	192.168.22.138	HTTP	258	GET /sfi9876.css HTTP/1.1
588724	193.468810	192.168.201.72	192.168.22.138	HTTP	251	GET /lpt9 HTTP/1.1
588725	193.469059	192.168.201.72	192.168.22.138	HTTP	258	GET /sfi9876.log HTTP/1.1
588745	193.478495	192.168.201.72	192.168.22.138	HTTP	258	GET /sfi9876.inc HTTP/1.1
588746	193.478516	192.168.201.72	192.168.22.138	HTTP	258	GET /sfi9876.dll HTTP/1.1

Figure 4: HTTP GET enumeration from 192.168.201.72 to 192.168.22.138, showing repeated requests for different file extensions and paths.

HTTP GET enumeration can be seen from Figure 4. Such behavior seems to be automated, as several requests were made simultaneously and the extensions of the files used were modified systematically. HTTP GET enumeration traffic consisted of keywords such as `sfish`, `skipfish.invalid`, and `Skipfish-Inject`, which shows that a web vulnerability scanner was used, such as Skipfish.

4.3 Finding 3: HTTP PUT Probing

The TCP stream that showed this request was PUT /PUT-sfi9876 HTTP/1.1 to 192.168.22.138, to which the server replied with HTTP/1.1 206 Partial Content. It is critical to note that having the PUT method enabled on the web server can be risky since the client would have the ability to upload/create files.

```
PUT /PUT-sfi9876 HTTP/1.1
Host: 192.168.22.138
Accept-Encoding: gzip
Connection: keep-alive
User-Agent: Mozilla/5.0 SF/1.71b
Range: bytes=0-199999
Referer: http://192.168.22.138/
```

```
HTTP/1.1 206 Partial Content
Date: Sat, 12 Mar 2011 19:04:53 GMT
Server: Apache/2.2.14 (Ubuntu)
X-Powered-By: MyBanco Customer Frontend v0.09
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Range: bytes 0-937/938
Content-Length: 938
Keep-Alive: timeout=15, max=88
Connection: Keep-Alive
Content-Type: text/html
```

Figure 5: TCP stream showing HTTP PUT probing against 192.168.22.138. The server responded with 206 Partial Content, so successful file creation is not confirmed.

In Figure 5, we see that the attacker tried to see whether the server can process PUT queries. The output does not contain 201 Created, 200 OK for storing a file, or any further fetching of a file which got uploaded successfully. Hence, this must be classified as PUT probing, and not successful file upload. However, regardless of whether the attack has been successful or not, it is noteworthy that the test was conducted since it suggests that the attacker was interested in examining the HTTP methods setup.

4.4 Finding 4: Web Application Payload Testing

The analysis demonstrated the presence of various types of web attacks in the HTTP request. `cmd.exe`, `/etc/passwd`, `/bin/sh`, SQL query, file redirection, a script tag, and an

alert JavaScript message were the parameters found. This indicates different vulnerabilities in the web application, such as command execution injection, local file inclusion, SQL injection, and XSS attacks.

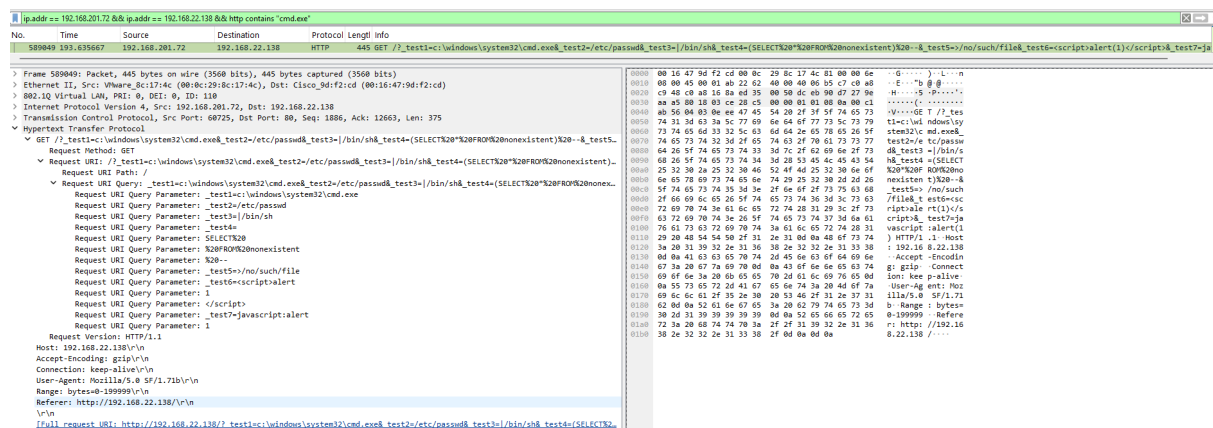


Figure 6: HTTP request containing command execution, local file inclusion, SQL injection, and cross-site scripting test payloads.

As shown in Figure 6, this represents automated vulnerability testing. Although there is no indication of any executed commands or the return of any sensitive files, there is an indication of the attacker attempting to test how the program processes dangerous input. The inclusion of paths for both Windows and Linux systems proves that the attacker did not know what configuration of server was being used and was testing many times. As the answer revealed, the server being used was Apache on Ubuntu, making `/etc/passwd` and `/bin/sh` payloads more useful.

4.5 Finding 5: Login Attack Attempts

Many HTTP POST requests are made for logging into the `/login/` URL too. This is seen in the captured session where some usernames like `admin` were entered along with some passwords. There are several examples where easily guessable passwords such as `admin` and `familyguy` were used. An example where password was set to `'or1=1` was also identified.

ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 && http.request.method == "POST"						
No.	Time	Source	Destination	Protocol	Length	Info
3746639	761.666987	192.168.201.72	192.168.22.138	HTTP	597	POST /login/ HTTP/1.1 (application/x-www-form-urlencoded)
> Frame 3746639: Packet, 597 bytes on wire (4776 bits), 597 bytes captured (4776 bits) > Ethernet II, Src: VMware_8c:17:4c (00:0c:29:8c:17:4c), Dst: Cisco_9d:f2:cd (00:16:47:9d:f2:cd) > 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 110 > Internet Protocol Version 4, Src: 192.168.201.72, Dst: 192.168.22.138 > Transmission Control Protocol, Src Port: 52962, Dst Port: 80, Seq: 1363, Ack: 11867, Len: 527 > Hypertext Transfer Protocol						
> POST /login/ HTTP/1.1\r\n Host: 192.168.22.138\r\n User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.9.2.13) Gecko/20101206 Ubuntu/10.10 (maverick) Firefox/3.6.13\r\n Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n Accept-Language: en-us,en;q=0.5\r\n Accept-Encoding: gzip,deflate\r\n Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\n Keep-Alive: 115\r\n Connection: keep-alive\r\n Referer: http://192.168.22.138/login/\r\n Content-Type: application/x-www-form-urlencoded\r\n Content-Length: 26\r\n \r\n [Response in frame: 3746649] [Full request URI: http://192.168.22.138/login/] File Data: 26 bytes						
> HTML Form URL Encoded: application/x-www-form-urlencoded > Form item: "user" = "admin" Key: user Value: admin > Form item: "pass" = "'or1=1" Key: pass Value: 'or1=1						

Figure 7: POST request to /login/ showing an SQL injection-style password value submitted with the username admin.

Figure 7 shows that the attacker entered inputs in the form on the login page. In this case, the attack was focused on authenticating, which made the attack worse compared to enumeration. It is suggested to employ both credential guessing and SQL injection testing techniques in this instance. If the application was vulnerable, then it could bypass authentication.

A later server response included HTTP/1.1 302 Found, Set-Cookie headers, and a redirect to /?login=true. This shows that the MyBanco application processed the login-related request and issued cookie-based state in response.

ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 && http contains "Set-Cookie"						
No.	Time	Source	Destination	Protocol	Lengtl	Info
9789315	1970.392226	192.168.22.138	192.168.201.72	HTTP	549	HTTP/1.1 302 Found
> Frame 9789315: Packet, 549 bytes on wire (4392 bits), 549 bytes captured (4392 bits) > Ethernet II, Src: Cisco_9d:f2:cd (00:16:47:9d:f2:cd), Dst: VMware_8c:17:4c (00:0c:29:8c:17:4c) > 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 110 > Internet Protocol Version 4, Src: 192.168.22.138, Dst: 192.168.201.72 > Transmission Control Protocol, Src Port: 80, Dst Port: 44232, Seq: 95323, Ack: 14423, Len: 479 > Hypertext Transfer Protocol - HTTP/1.1 302 Found\r\n - Response Version: HTTP/1.1 - Status Code: 302 [Status Code Description: Found] - Response Phrase: Found - Date: Sat, 12 Mar 2011 19:34:30 GMT\r\n - Server: Apache/2.2.14 (Ubuntu)\r\n - X-Powered-By: MyBanco Customer Frontend v0.09\r\n - Set-Cookie: user=.%5Cskipfish; expires=Thu, 24-Mar-2011 19:34:30 GMT; path=/\r\n - Set-Cookie: id=deleted; expires=Fri, 12-Mar-2010 19:34:29 GMT; path=/\r\n - Location: /?login=true\r\n - Vary: Accept-Encoding\r\n - Content-Encoding: gzip\r\n - Content-Length: 20\r\n [Content length: 20] - Keep-Alive: timeout=15, max=25\r\n - Connection: Keep-Alive\r\n - Content-Type: text/html\r\n \r\n [Request in frame: 9789275] [Time since request: 33.305000 milliseconds] [Request URI: /login/] - Content-encoded entity body (gzip): 20 bytes -> 0 bytes - File Data: 0 bytes						

Figure 8: HTTP 302 Found response from 192.168.22.138 showing cookie handling and redirection to /?login=true.

This is not necessarily indicative of successful authentication, since redirection and cookies can be used for the process of logging in or failing to log in. Due to the fact that the capture doesn't show a dashboard, account, or other clear indication of success, the findings will be reported as attacks on login with session handling, rather than compromised accounts.

4.6 Finding 6: Transaction Endpoint Testing

The suspected attacker also targeted transfer-related endpoints, including /transfer/account/ and /transfer/account/doTransaction. This conclusion is critical since the endpoint's title suggests its importance to the business. POST requests contained arguments such as code=000 and transactionID=.

```
POST /transfer/account/doTransaction HTTP/1.1
Host: 192.168.22.138
Accept-Encoding: gzip
Connection: keep-alive
User-Agent: Mozilla/5.0 SF/1.71b
Range: bytes=0-199999
Referer: http://192.168.22.138/
Content-Type: application/x-www-form-urlencoded
Content-Length: 23
```

```
code=000&transactionID=
HTTP/1.1 206 Partial Content
Date: Sat, 12 Mar 2011 19:31:10 GMT
Server: Apache/2.2.14 (Ubuntu)
X-Powered-By: MyBanco Customer Frontend v0.09
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Range: bytes 0-936/937
Content-Length: 937
Keep-Alive: timeout=15, max=44
Connection: Keep-Alive
Content-Type: text/html
```

Figure 9: TCP stream showing a POST request to `/transfer/account/doTransaction` with submitted transaction parameters. The server responded with `206 Partial Content`, confirming endpoint interaction but not successful transaction execution.

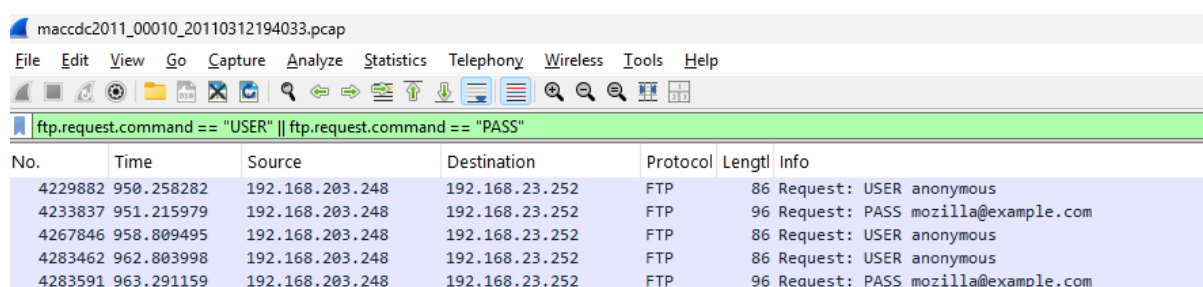
Figure 9 depicts the interaction with the transaction endpoint. Other requests included values like `transactionID=9876sfi`, `code=2147483647`, `code=000'true'`, and `code=000'`uname`'`. These values indicate parameter fuzzing and injection testing against transaction logic. This could suggest an attempt to avoid transaction validation, initiate command execution, or detect poor input handling.

The server returned `206 Partial Content` status code meaning that the requests were received by the web application. Nevertheless, there are no signs of the transaction being conducted successfully, of the account balance changing, and of an unauthorized funds transfer. The appropriate inference from the above is that the transaction logic was

successfully researched, yet no transaction abuse was confirmed.

4.7 Finding 7: Anonymous FTP Login Attempts

FTP traffic was also identified between 192.168.203.248 and 192.168.23.252. It seems the client sent `USER anonymous` and `PASS mozilla@example.com`. These values were visible in plain-text, showing the weakness of FTP as a protocol for authentication and file transfer (Postel & Reynolds 1985). This finding was cross-checked in NetworkMiner, which extracted the same FTP authentication artifacts from the filtered FTP PCAP, as shown in Figure 1.



The image shows a Wireshark capture of FTP traffic. The filter bar at the top is set to `ftp.request.command == "USER" || ftp.request.command == "PASS"`. The packet list below shows five packets, all of which are FTP requests from 192.168.203.248 to 192.168.23.252. The first and third packets are `Request: USER anonymous`, and the second, fourth, and fifth packets are `Request: PASS mozilla@example.com`. The packet details pane on the right shows the raw data for the selected packet, which is the `PASS` command.

No.	Time	Source	Destination	Protocol	Length	Info
4229882	950.258282	192.168.203.248	192.168.23.252	FTP	86	Request: USER anonymous
4233837	951.215979	192.168.203.248	192.168.23.252	FTP	96	Request: PASS mozilla@example.com
4267846	958.809495	192.168.203.248	192.168.23.252	FTP	86	Request: USER anonymous
4283462	962.803998	192.168.203.248	192.168.23.252	FTP	86	Request: USER anonymous
4283591	963.291159	192.168.203.248	192.168.23.252	FTP	96	Request: PASS mozilla@example.com

Figure 10: Wireshark FTP filter showing anonymous FTP login attempts from 192.168.203.248 to 192.168.23.252. The credentials are visible in plaintext.

Figure 10 shows FTP login attempts made anonymously. From the FTP flow, the server returned `230 User successfully logged in`, indicating that anonymous access was allowed. Following the login process, commands such as `MKD`, `CWD`, and `LIST` were observed with abnormal parameters. This suggests an automated FTP scan or fuzzing operation. The challenge here is that anonymous FTP access could enable file manipulation based on server constraints.

5 Security Threats and Impact

PCAP shows multi-steps of attacks. SYN scanning implies reconnaissance within the internal network. HTTP traffic shows the attempt to scan web applications for 192.168.22.138. The login attempts show credential guessing and probably SQL injection test. Transaction endpoint traffic shows the penetration into sensitive application features. FTP traffic shows unsecured, anonymous connections to internal services.

The main vulnerabilities can be considered open internal services, insufficient protection of web applications' inputs, exposed authentication channels, and unsecured FTP connection. Since HTTP and FTP connections were established in clear text, the hacker did not

have to use any encryption or covert techniques to interact with the target system. This allowed more visibility for the analyst; however, it also implied that any other person who had access to the network could intercept the credentials and application-related requests. The confirmed effect consists of the success of anonymous FTP logins and numerous web application scans. Account hijacking, any illegal transactions, file disclosure, and further reconnaissance can be among the consequences. Still, the provided data does not confirm any online attack or full transaction execution. The difference is essential because, although the PCAP provides evidence of attack attempts, it does not confirm the success of each attempt made (Cichonski et al. 2012).

A high volume of automated scans might impact the work of MyBanco’s application. If the target transaction endpoint has been successfully attacked, some financial and account details could be modified. Moreover, the use of FTP service is highly risky because anonymous FTP access could lead to file disclosure or even file staging.

6 Critical Discussion

Alternatively, the attacker could have refined their attack through the use of the findings from the SYN scan to target specific vulnerabilities instead of relying on a generalized automated attack. Moreover, the attacker would have taken precautions against detection through slowing down their scan, escalated privileges after gaining access, and used the FTP service that succeeded to upload files. In case the login bypass had succeeded, the attacker would have been able to access authentication screens and manipulate transactions.

Various suggestions are made for mitigating these attacks. Anonymity in FTP transfers is to be disabled forthwith and substituted by SFTP and FTPS. FTP services need to be accessed using named accounts, strong passwords, and network constraints. PUT method, which is not required at present, is to be disabled until needed. Input validation, output encoding, query parameters, and CSRF defense are needed in the MyBanco application. Rate limitation and account locking need to be enforced on login pages (OWASP Foundation 2024, 2021).

Detection must also become better. Alarms need to be raised where there is a large volume of SYN attacks, repeated HTTP requests with abnormal extensions, unusual user agent strings like `SF/1.71b`, and payloads including `/etc/passwd`, `cmd.exe`, `<script>`, and `Skipfish-Inject`. There must be network segregation to allow host-to-host communication between hosts inside. Having a centralized log system and monitoring the logs from the web servers will also help. These payload categories are consistent with common web application risks such as injection, broken input handling, and cross-site scripting,

which are highlighted in OWASP guidance (OWASP Foundation 2021, 2024).

7 Conclusion

This PCAP offers concrete evidence of malicious behavior on the network. This includes SYN scans, HTTP probes, PUT probes, web application attack payloads, authentication attempts, transaction endpoints probing, and FTP access anonymously. The strongest evidence of a breach is that anonymous FTP was permitted between 192.168.203.248 and 192.168.23.252. Even though there have been many attacks on 192.168.22.138, there is no strong evidence of a successful attack against the web application or transactions.

In conclusion, the incident illustrates significant vulnerabilities, including internal open services, FTP without proper protection, applications susceptible to attacks, and lack of defense against automatic web attacks.

References

- Cichonski, P., Millar, T., Grance, T. & Scarfone, K. (2012), ‘Computer Security Incident Handling Guide’, NIST Special Publication 800-61 Revision 2. Available at: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>. Accessed: 11 May 2026.
- Netresec (2025), ‘NetworkMiner: Network Forensic Analysis Tool’, Available at: <https://www.netresec.com/?page=NetworkMiner>. Accessed: 11 May 2026.
- OWASP Foundation (2021), ‘OWASP Top 10: Web Application Security Risks’, Available at: <https://owasp.org/www-project-top-ten/>. Accessed: 11 May 2026.
- OWASP Foundation (2024), ‘Web Security Testing Guide’, Available at: <https://owasp.org/www-project-web-security-testing-guide/>. Accessed: 11 May 2026.
- Postel, J. & Reynolds, J. (1985), ‘File Transfer Protocol’, RFC 959. Available at: <https://www.rfc-editor.org/rfc/rfc959>. Accessed: 11 May 2026.
- Wireshark Foundation (2025), ‘Wireshark User’s Guide’, Available at: <https://www.wireshark.org/docs/>. Accessed: 11 May 2026.

A Appendix A: Key Wireshark Filters

```
ip.src == 192.168.201.74 && tcp.flags.syn == 1 && tcp.flags.ack == 0
```

`ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 && http.request`

`ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 &&
http.request.method == "PUT"`

`ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 &&
http contains "cmd.exe"`

`ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 &&
http.request.method == "POST"`

`ip.addr == 192.168.201.72 && ip.addr == 192.168.22.138 &&
http contains "doTransaction"`

`ftp.request.command == "USER" || ftp.request.command == "PASS"`

`ip.addr == 192.168.203.248 && ip.addr == 192.168.23.252 && ftp`